Building ASP (Agent Semantic Protocol): A Structure-Aware Document RAG Approach for the Enterprise

Author: Kushal Sharma **Date:** 2026-06-26

1. Executive summary

- **The state of document RAG in 2026 is monoculture.** Every major cloud (Azure, AWS, Google, Snowflake, Databricks), every major vector database (Pinecone, Weaviate, Elastic), and the dominant legal and enterprise AI products (Glean, CoCounsel, Harvey, NetDocuments) ship variants of the same recipe: cut documents into fixed-size chunks, index them with a hybrid of BM25 and dense vectors, and rerank.
- **The "hierarchical RAG" label is doing a lot of work it does not earn.** Most things sold as hierarchical are either size-based parent/child token windows (AWS Bedrock), summary trees built post-hoc by clustering (RAPTOR, GraphRAG community detection), or layout-aware chunking that still feeds a flat vector index (Azure Document Layout, Docling, Unstructured). None of these is the document's own structure.
- **A small set of methods do retrieve directly over a document's native heading and section tree** — notably PageIndex, HiQA, RDR2, LAD-RAG, and Google's layout-aware mode inside Vertex AI Search — but none is a mainstream production default in 2026, and the most production-ready option among them sits behind a premium Google Cloud SKU at roughly **$4 per 1,000 enterprise search queries**, plus an additional LLM grounding charge for RAG-style answers. *(Vertex AI Search publishes per-query pricing across Standard and Enterprise tiers, with an additional LLM Add-On for grounded generation. Confirm the exact figure for the current quarter on the Vertex AI Search pricing page before quoting externally; pricing has been restructured across multiple rebrands (Discovery Engine → Vertex AI Search → Gemini Enterprise) and is best read from the live page on the day of the talk.)*
- **ASP — the *Agent Semantic Protocol* — is being built to occupy that gap.** It retrieves and assembles in the document's native coordinate system — parsed tree, heading-path BM25, parent/sibling expansion, authority-ordered context bundles with structural headers — instead of over fixed-size chunks, and it runs on self-hosted infrastructure rather than a metered SaaS endpoint.
- **One representative experiment in this report — EE4-X1, the *Adaptive Context Floor*** — illustrates what structure-aware retrieval enables in practice: it expands an undersized bundle by walking the document's own parent and sibling edges until a minimum-context floor is met. On the MAUD legal multiple-choice benchmark it converted a measurable deficit into a measurable lead against the flat-hybrid baseline. On the CUAD contract-extraction benchmark it fires more eagerly than it should and mildly dilutes spans — a known gating limitation that the next experiment addresses.

2. Why document structure matters

Document RAG is the problem of answering a question by retrieving evidence from one or more long, structured documents — contracts, policies, regulatory filings, multi-section reports — and presenting that evidence to a language model.

The lowest-common-denominator solution is to chop each document into fixed-size token windows, embed each window into a vector, and at query time return the windows whose embeddings sit closest to the query. It is cheap, generic, and works tolerably on short corpora. It is also the wrong shape for the documents enterprises actually run on. A 200-page credit agreement is not a bag of 500-token chunks. It is a tree — *Article* → *Section* → *Sub-clause* → *Paragraph* — and the answer to almost every business question depends on where in that tree the evidence sits, what sits next to it, and which source it came from.

The positioning of this work is simple. **Retrieval that operates on the document's parsed structure should outperform retrieval that ignores it, and it should do so on infrastructure the enterprise owns.** Section 3 surveys what the market is shipping today against that bar. Section 4 sets out where ASP differs. Sections 5–6 show the supporting evidence.

---

3. The 2026 landscape — what enterprises run today

The methods deployed and discussed in 2026 fall into five categories. Each is summarised below with named methods, primary citation, and an explicit note on whether the method uses the document's native heading/section structure as the retrieval signal — as opposed to using structure only as a parsing or chunking heuristic.

3.1 Flat chunking + vector retrieval — the de-facto baseline

The default RAG pipeline in 2026 is fixed-size token windowing followed by dense vector retrieval. Pinecone advises that *"fixed-sized chunking will be the best path in most cases."* Weaviate recommends a 512-token / 50-100-overlap baseline. Snowflake Cortex Search caps chunks at 512 tokens. Azure AI Search's default Text Split skill uses 2,000-char pages with 500-char overlap. ColBERT and ColPali sit one layer below as passage-level and page-image alternatives but do not change the underlying chunking premise. **Native structure used: no.**

3.2 Hybrid retrieval — BM25 + dense vector + reranker

The enterprise default upgrades 3.1 by running BM25 in parallel with a dense vector retriever, fusing the two ranked lists (typically with Reciprocal Rank Fusion), and reranking the top candidates with a cross-encoder. This pattern was popularised by BEIR and is now the implementation behind Databricks Mosaic AI Vector Search, Elastic semantic_text + ELSER, Snowflake Cortex Search, Glean enterprise search, Anthropic Contextual Retrieval, and the LangChain and LlamaIndex reference stacks. **Native structure used: no — chunks are still flat, and structure does not enter the ranker.**

3.3 Synthetic-hierarchy and clustering-based methods — "hierarchical-looking" but built post-hoc

These methods construct a tree, but the tree they build is not the document's tree.

- **RAPTOR** (Sarthi et al., ICLR 2024) recursively embeds, clusters, and summarises chunks bottom-up.
- **LlamaIndex DocumentSummaryIndex** creates a two-level *document-summary* → *chunks* construct.
- **LangChain ParentDocumentRetriever** splits parents and children using a recursive character text splitter.
- **AWS Bedrock Hierarchical Chunking** is a 1,500-token-parent / 300-token-child token-window scheme.

Native structure used: no — the hierarchy is synthesised from text, not lifted from the document.

3.4 Graph-based RAG — entity graphs, not document graphs

- **Microsoft GraphRAG** slices into TextUnits, LLM-extracts entities and relations, and Leiden-clusters the resulting entity graph into community summaries.
- **LightRAG** and **HippoRAG** run LLM OpenIE over chunks.
- **Neo4j GraphRAG SimpleKGPipeline** performs schema-guided LLM extraction.
- **KAG** (Ant Group) is the closest hybrid — it adds layout analysis to mutual chunk-to-graph indexing — but the graph remains schema- and LLM-driven.

Native structure used: no — the graph is over entities the LLM extracted, not over the document's sections.

3.5 Native-structure retrieval — the rare genuine cases

A small set of methods do retrieve over the document's parsed tree as the primary signal:

- **PageIndex** (VectifyAI, 2024–2025) builds a TOC-like tree and navigates it by LLM reasoning. Vector-free. Per-query latency of 30–120 seconds keeps it in research and proof-of-concept use.
- **HiQA** (Chen et al., 2024) parses sections and cascades the heading metadata into each chunk before retrieval, so the vector retriever sees the hierarchy.
- **RDR2** (Xu et al., EMNLP 2025 Findings) trains a router that navigates parsed document structure trees as a primary retrieval task.
- **LAD-RAG** constructs a layout graph of visually-rich documents and retrieves against it.
- **Google Vertex AI Search — layout-aware chunking mode** is the closest production case: chunks are bounded by layout entities and can prepend ancestor headings, and the service is offered behind a premium per-query SKU with an additional LLM grounding charge.

Layout-aware parsers — LlamaParse, Azure AI Document Intelligence Layout, Docling HierarchicalChunker, Unstructured's `chunk_by_title`, Marker, and Adobe PDF Extract — preserve heading structure in metadata but emit a flat chunk stream that the downstream retriever still treats as a bag. **Structure-aware chunking is not the same as structure-aware retrieval.**

3.6 Reading the landscape

The picture is consistent across all five categories. Mainstream production stacks and the open-source frameworks that feed them retrieve over flat, chunked representations. The structure of the document is, at best, used to draw the boundaries of the chunks; it is not the signal the retriever ranks against. The methods that do treat the document's parsed tree as a first-class retrieval surface are concentrated in late-2024 and 2025 research releases and in one premium SaaS SKU. **There is open ground between research and production for a structure-aware document RAG approach that an enterprise can own and operate on its own infrastructure.**

---

4. Positioning of ASP

ASP is being built to occupy that open ground. Its design commitments are deliberately different from the 2026 mainstream:

1. **Native-tree retrieval, not chunking.** The parsed document tree — articles, sections, sub-clauses, paragraphs — is the unit of retrieval, ranking, and citation. There is no fixed-size token window in the index.
2. **Structural bundle assembly, not concatenate-and-truncate.** Context is assembled from tree nodes in an order that respects source authority, with explicit token budgeting, structural headers, and an audit trail of anything that was dropped.
3. **Self-hosted, not metered.** ASP runs on infrastructure the enterprise controls. It does not depend on a per-query SaaS endpoint whose unit economics scale linearly with usage. The closest production-ready structure-aware alternative — Vertex AI Search with the LLM Add-On — is offered at roughly $4 per 1,000 enterprise-tier queries plus the grounding charge, which compounds quickly for a workflow that issues thousands of retrieval calls per day.
4. **Same-condition benchmarking against the prevailing pattern.** Every claim in this work is measured against a fair flat-hybrid baseline (BM25 + dense vector + cross-encoder reranker) running in the same process, on the same questions, on the same documents.

How ASP retrieves

- **Parsing.** A dispatcher detects numbering and heading conventions and emits a hierarchical *DocumentTree* — articles, sections, sub-clauses, paragraphs. Where the dispatcher cannot resolve a structure, the pipeline falls back to paragraph-split section nodes so the downstream contract is preserved. Multi-document corpora can supply pre-built node dictionaries.
- **Node.** A *TreeNode* addresses one structural unit. It carries a stable `node_id`, `node_type`, `doc_slug`, title, full `heading_path` (e.g. *"Article 7 > Conditions to Closing"*), depth, content, optional `clause_id`, parse confidence, and token/char counts, with parent and child links so the tree is navigable up and sideways. **The node — not a fixed-size chunk — is the unit of retrieval, ranking, and citation.**
- **Retrieval traversal.** (i) Path-augmented BM25 scores every node, weighting matches in the heading path as well as the body, with a depth multiplier tuned to whether the corpus favours leaf clauses or section paragraphs. (ii) A candidate-seeding step selects high-scoring nodes while ensuring multi-source coverage so a single dominant document cannot monopolise the set. (iii) A tree-traversal expansion walks ancestors and siblings of the seeds to add structurally adjacent context that pure BM25 would miss. (iv) A cross-encoder reranks the expanded set and retains the top-K.
- **Bundle assembly.** The surviving nodes are assembled into an ASP *ContextBundle*. Nodes are sorted first by source `authority_level` and only secondarily by retrieval rank, so authoritative sources lead the prompt. For each node, a paragraph extractor pulls the query-relevant paragraphs, and a compact header is emitted naming the source, the leaf section, and the clause ID — `[<SourceName> <LeafSection> | <ClauseID>]`. Tokens are budgeted explicitly against a hard cap; if anything is dropped, the bundle records `compression_applied` and surfaces what was dropped. The protocol forbids silent truncation.

Contrast with §3

The flat baseline indexes fixed-size token windows and assembles context by string concatenation plus truncation, with no awareness of section, clause, or source authority. ASP retrieves and assembles in the document's native coordinate system. Synthetic-hierarchy methods (RAPTOR, GraphRAG, LlamaIndex DocumentSummaryIndex) build a tree but not the document's tree. Layout-aware parsers (LlamaParse,

Docling, Vertex's layout chunker) preserve structure as metadata but the downstream retriever still runs over a flat list. ASP is closest in spirit to PageIndex and RDR2 — primary retrieval over a parsed native tree — but uses BM25 + tree-expansion + cross-encoder rather than LLM tree-walk, which sidesteps PageIndex's latency problem.

ASP carries additional, orthogonal experiments at the prompt and seeding layers; those are out of scope for this report.

---

5. A representative experiment — *Adaptive Context Floor* (EE4-X1)

One experiment illustrates what becomes possible once retrieval operates on the document's structure rather than on a flat chunk stream.

5.1 The problem this experiment addresses

Some single-source classification and clause-comparison tasks fail not because retrieval picks the wrong clause, but because retrieval picks the *right* clause in isolation. The model sees one ~190-token clause and cannot distinguish near-neighbour candidates (for example, *"Mixed Cash/Stock"* versus *"Mixed Cash/Stock: Election"*) because the discriminating context lives in the *sibling* clauses it never saw.

We refer to this failure mode as **context starvation**. The hypothesis is that lifting the bundle above a minimum-context floor — by adding the parent section and its other children — closes the gap without any help from BM25 scores and without any dataset-specific signal.

5.2 Method

A `min_context_tokens` floor was added to the bundle assembler. After the normal authority-sorted, budget-bounded assembly pass, if the assembled bundle is below the floor and the full node graph is available, the assembler runs a structure-only BFS expansion: it enqueues the parent of each included node, then the parent's other children, and adds nodes until the floor is met or the hard token budget would be exceeded. Expansion is guided exclusively by tree topology — no BM25 scores, no query re-analysis, no dataset flags. The floor is plumbed through the pipeline as `min_context_tokens` and exposed on the runner as `--min-context-tokens`. The benchmark sweep ran at `min_context_tokens=700`.

The primary code change is in `src/asp/benchmarks/retrieval/bundle_assembler.py`: a new `_expand_to_floor()` helper (BFS over parent then siblings, budget-respecting) and two new parameters on `assemble_context_bundle` (`min_context_tokens`, `nodes_dict`).

5.3 Gating — and its current limit

The expansion has no explicit dataset gate. The only built-in gate is implicit: the floor fires only when the post-budget bundle is below `min_context_tokens`. On multi-source corpora — where the assembled bundle is already well above 700 tokens — the floor never fires, so the mechanism is a no-op. The mechanism therefore activates exactly where it was designed to: on single-source corpora whose retrieval converges to a small number of short clauses.

The empirical result confirmed this gate is too permissive in one case: CUAD is single-source with short bundles (~240 tokens), so the floor fires — but CUAD is a span-extraction task where adding parent and sibling clauses dilutes the precise extraction span. The next experiment in the sequence (EE4-X2) replaces the implicit gate with a sibling-BM25-score-variance test, so the floor only triggers when retrieval is genuinely

non-discriminating.

5.4 Principle

> When the assembled bundle is too small to distinguish a candidate from its near-neighbours, expand along the document's own structural edges — parent first, then siblings — until the model has enough surrounding clauses to discriminate, never exceeding the hard token budget.

5.5 Why this is a useful demonstration

This is a direct demonstration that document-tree structure can be used at bundle-assembly time to repair a failure mode that a flat-chunk retriever cannot even see, let alone fix. A chunk retriever has no parent and no siblings; ASP does.

---

6. Results

The flat baseline in these benchmarks is `flat_hybrid_rag.py`: BM25 + dense vector (Chroma, BAAI/bge-small-en-v1.5) fused with Reciprocal Rank Fusion (k=60), reranked with `cross-encoder/ms-marco-MiniLM-L-6-v2`, K_retrieve=10 per arm, K_rerank=5. This is the same retrieval pattern enterprises deploy today (the BEIR / LangChain / LlamaIndex hybrid default), so the ASP-versus-flat comparison is a fair stand-in for *ASP versus what the enterprise actually runs.*

`n=500` per dataset for every cell. Numbers rounded to 1 decimal.

6.1 Baseline run (no context floor) — `rr_final`

Dataset	Model slot	Metric	ASP	Flat	Δ
CUAD	m2 (Nem)	EM	87.7	69.9	+17.8
CUAD	m2 (Nem)	F1	91.6	74.4	+17.2
CUAD	m4 (Mis)	EM	87.1	69.3	+17.8
CUAD	m4 (Mis)	F1	91.5	74.9	+16.5
CUAD	m5 (Nem)	EM	86.8	70.0	+16.8
CUAD	m5 (Nem)	F1	90.9	75.2	+15.8
CUAD	m6 (Mis)	EM	88.6	69.9	+18.7
CUAD	m6 (Mis)	F1	92.5	74.5	+18.0
MAUD	m3 (Nem)	Acc	36.2	39.4	–3.2
MAUD	m4 (Mis)	Acc	37.8	41.6	–3.8
HotpotQA	m1 (Nem)	F1	64.3	69.7	–5.4
HotpotQA	m2 (Mis)	F1	66.2	71.7	–5.6

HotpotQA	m3 (Nem)	F1	65.4	69.4	–4.0
HotpotQA	m5 (Nem)	F1	65.9	70.9	–5.0
EW1	m5 (Nem)	F1	11.8	7.6	+4.2
EW1	m6 (Mis)	F1	9.6	8.9	+0.7
EW2	m5 (Nem)	F1	10.6	8.1	+2.5
EW2	m6 (Mis)	F1	10.9	9.1	+1.8

6.2 Adaptive Context Floor active — `rr_ctx700`

Dataset	Model slot	Metric	ASP	Flat	Δ
CUAD	m5 (Nem)	EM	81.0	65.4	+15.6
CUAD	m5 (Nem)	F1	85.4	71.0	+14.4
CUAD	m6 (Mis)	EM	78.8	64.8	+14.0
CUAD	m6 (Mis)	F1	83.1	69.4	+13.7
MAUD	m3 (Nem)	**Acc**	**36.2**	**32.0**	**+4.2**
MAUD	m4 (Mis)	**Acc**	**36.4**	**35.2**	**+1.2**
EW1	m5 (Nem)	F1	7.7	5.4	+2.3
EW1	m6 (Mis)	F1	7.8	5.4	+2.4
EW2	m5 (Nem)	F1	10.5	7.8	+2.6
EW2	m6 (Mis)	F1	11.1	8.6	+2.5

HotpotQA is not included in the `ctx700` row because the assembled bundle already sits well above the 700-token floor (~1,200 tokens). The floor is a no-op there and the run was not repeated.

6.3 Reading the tables

- **MAUD flips from trailing to leading.** Under the baseline, ASP trailed flat by 3.2 / 3.8 points on the two model slots. Under the context floor, ASP leads by 4.2 / 1.2 points. Among the 345 of 500 rows where expansion actually fired, accuracy rose 2.9 / 1.4 points — a clean causal signal on the dataset the mechanism was designed for.
- **CUAD remains a clear lead but loses some of its ceiling.** ASP still beats flat by 13–16 points EM/F1, but the absolute numbers drop from ~87 EM in the baseline to ~80 EM with the floor active. This is the gating limitation described in §5.3: CUAD's short, span-extraction bundles trigger the floor, and the added parent and sibling clauses dilute the extraction span. The follow-up experiment (a sibling-score-variance gate) is designed to leave CUAD untouched.
- **EW1 and EW2 retain a mild positive.** Both are long-form extraction where Exact Match is expected to be 0; ASP's F1 advantage over flat is preserved under the floor.

- **HotpotQA is unchanged by EE4 (no-op).** The baseline HotpotQA gap is a separate problem addressed by other experiments out of scope for this report.
- **Bundle size and latency.** On the single-source benchmarks, ASP context bundles remain dramatically smaller than the flat baseline (CUAD: 240 vs 682 tokens; MAUD with the floor: 825 vs 870; EW1 with the floor: 966 vs 1,283). On the `rr_ctx700` run, ASP latency was 4–63× lower than the flat baseline across single-source datasets. (Caveat: the `rr_ctx700` run executed in a different inference environment from `rr_final`, so latency improvements versus the prior ASP run should not be attributed to EE4-X1 itself — the ASP-versus-flat ratio *within* the same run is the load-bearing comparison.)

---

7. Open work

The Adaptive Context Floor establishes that structural expansion at bundle-assembly time can repair context starvation on MAUD. Three threads remain open.

1. **CUAD dilution.** The floor currently fires whenever the bundle is short. The follow-up (EE4-X2) replaces that with a sibling-BM25-score-variance gate so the floor only triggers when retrieval is non-discriminating — the MAUD failure mode — and leaves CUAD's already-discriminating retrievals untouched.
2. **Floor calibration.** The 345-of-500 fire rate on MAUD is informative but not yet tuned. Whether 700 tokens is the right floor — or whether it should adapt to candidate-clause length — is an open calibration question.
3. **Generalisation.** The mechanism has been validated on one single-source legal MC corpus. Whether it transfers to other near-neighbour classification problems (regulatory categorisation, policy compliance triage) is the natural next benchmark.

---

8. Appendix — survey matrix

The full set of methods reviewed in §3, with each method's category, whether it uses native document structure as the retrieval signal, and a primary source.

Name	Category	Native structure?	Source
Hybrid BM25 + Dense RAG	Flat hybrid	No	BEIR, Thakur et al. 2021
Anthropic Contextual Retrieval	Flat hybrid + LLM chunk context	Partial (context prefix, no tree)	anthropic.com
Azure AI Search (Text Split)	Cloud platform	No	learn.microsoft.com
Azure AI Search (Document Layout)	Cloud platform	Partial (structure → chunk boundaries)	learn.microsoft.com
AWS Bedrock KB — Hierarchical Chunking	Cloud platform	No (token parent/child)	docs.aws.amazon.com

Google Vertex AI Search — Layout-Aware	Cloud platform	Yes (premium SKU, metered per query)	cloud.google.com
Snowflake Cortex Search	Cloud platform	No	docs.snowflake.com
Databricks Mosaic AI Vector Search	Cloud platform	No	databricks.com
Pinecone	Vector DB	No	pinecone.io
Weaviate	Vector DB	No	weaviate.io
Elastic / ELSER	Search engine	No	elastic.co
ServiceNow AI Search	Enterprise search	Partial (table-rooted)	servicenow.com
Glean Enterprise Search	Enterprise search	Partial (entity graph)	glean.com
Harvey AI Vault	Legal AI	Partial (undisclosed)	harvey.ai
Hebbia Matrix	Agentic doc QA	Partial (page/section iteration)	hebbia.com/matrix
Casetext CoCounsel	Legal AI	No	casetext.com
NetDocuments ndMAX	Legal DMS AI	No	netdocuments.com
RAPTOR	Research	No (synthetic tree by clustering)	arXiv:2401.18059
GraphRAG (Microsoft)	Research / graph	No (entity graph, Leiden community hierarchy)	arXiv:2404.16130
LightRAG	Research / graph	No (LLM OpenIE over chunks)	arXiv:2410.05779
HippoRAG	Research / graph	No (LLM OpenIE + PPR)	arXiv:2405.14831
KAG (Ant Group)	Research / graph	Partial (layout + LLM extraction)	arXiv:2409.13731
Neo4j GraphRAG SimpleKGPipeline	OSS graph	No	neo4j.com
LlamaIndex PropertyGraphIndex	OSS graph	Partial (configurable)	llamaindex docs
LangChain GraphRetriever	OSS graph	Partial (metadata edges)	langchain docs
LlamaIndex DocumentSummaryIndex	OSS hierarchical	No (synthetic summary tree)	llamaindex docs
LangChain ParentDocumentRetriever	OSS hierarchical	No (size split)	langchain docs
Dense X Retrieval (Propositions)	Research	No	arXiv:2312.06648

HiRAG with Rethink	Research	Partial (coarse-to-fine scaffold)	arXiv:2408.11875
MultiHop-RAG	Research / benchmark	No	arXiv:2401.15391
HIRAG (Instruction Tuning)	Research	No (hierarchy in reasoning, not index)	arXiv:2507.05714
ColBERT	Research	No (passage-level)	arXiv:2004.12832
ColPali	Research	No (visual patches)	arXiv:2407.01449
PageIndex	Native tree	**Yes**	github.com/VectifyAI/PageIndex
HiQA	Native tree (cascaded)	**Yes**	arXiv:2402.01767
RDR2	Native tree	**Yes**	arXiv:2510.04293
LAD-RAG	Native layout graph	**Yes**	arXiv:2510.07233
RAG with Hierarchical Knowledge	Native tree	**Yes**	arXiv:2503.10150
MacRAG / Multi-granularity	Research	Partial (granularities from structure)	representative arXiv
LlamaParse	Parser	Partial (metadata only)	llamaindex docs
Azure AI Document Intelligence Layout	Parser	No (feeds flat retriever)	learn.microsoft.com
Unstructured.io (`chunk_by_title`)	Parser	No (flat output)	unstructured docs
Docling HierarchicalChunker	Parser	Partial (metadata only)	docling-project.github.io
Marker	Parser	Partial (tree available, RAG path flat)	github.com/datalab-to/marker
Nougat	Parser	No	github.com/facebookresearch/nougat
pdfplumber	Parser	No	github.com/jsvine/pdfplumber
Adobe PDF Extract API	Parser	No (feeds caller's retriever)	developer.adobe.com
ASP (this work)	Native tree retrieval + structural bundle assembly	**Yes**	—